The machines

The machines

Graal Analysis, revision 0.0.2.

A person who logs in meets other programs before they meet a world. The client asks a directory. On the classic listserver that link closes once the list and a status sentence have been sent. A second program, the game server, then decides whether this account and this build may walk. Somewhere in that world a third program may already be logged in, owning the scripts. Staff watch the same world through a window that grew, over two years, from an account list into a file browser. The levels those people walk were drawn by tools that never open a socket.

This chapter is that path. The login line, the walk message, and the board packet are in chapter 1. The body is in chapter 2. Byte layouts stay in the notes cited beside each machine.

The observations were made on loopback. The capture lab puts each client in a network namespace whose only interface is the loopback address, with a private hosts file and a separate display, and it keeps the packet files on the lab machine (`tools/lab-setup.md`). Recovered programs were read and not executed (`recovered-hacks/behaviour-models.md`, `tools/classic-tools/README.md`).

The directory that answers first

The classic listserver is one process with two listen ports, one for clients and one for game servers, and a database for accounts and the public directory (`tools/classic-tools/serverlist3.md`). `[bin]` It is the ELF from the Server Lister package. It is a different program from the 2003 Windows `ListServer.exe` below.

It starts only if it is given the full path of a config file. A missing file produces two console lines whose spellings are in the image, `Configuraton not found.` and `Pleace specify the full path`…, and then the process exits. `[bin]` The keys it reads are the client port, the server port, a player limit, the database server name, and a show-more URL. A sample file in the same package sets the two ports to the lab client ports. `[file]` Beside itself it writes four text files: a player count, a max, an account count, and a stats file that marks players upgraded, ghost, or trial.

A client has two useful things to say, in that program's own words. An edition line selects directory rows whose edition contains that text and whose last write is under ten minutes. An account line loads the account by name. The password is checked locally, as a hash of the first eight characters, and it is not placed in the query. Any other line is ignored. There is no later cipher and no version token of the kind the game server will demand. `[bin]` Chapter 1 records three zlib shapes for what 2.00, 2.05, 2.17, 2.22 and 2.31 send to a listserver. Chapter 4 sets those shapes beside this edition line and this account line. This chapter leaves them as the notes wrote them.

What the client is shown is short, and then the link closes. A hard failure is an error and a close: the password is wrong, the account is banned, or the account does not exist yet. A success is the directory and a status sentence. The status can welcome the player, speak a staff sentence, count remaining time, or explain a trial or a free server. The trial sentence and the free-server sentence also carry the show-more URL from the config. The process does not open a browser and does not read a saved login. `[bin]`

Each directory row is a name, a language, a description, a homepage, a version, a player count, an address, and a port. `[bin]` Those rows come from the database table. The game server's own messages do not build them.

A game server's first line, on this listserver, is a kind and an eight-byte token, `GLSER001` or `GLSER002`, and then the world name. Any other kind or token closes the link. The world name is looked up, and the socket's address has to match the address stored for that world, or the console says the server is not allowed to connect from its address. `[bin]` After that login the listserver acts on two messages: a player arriving or changing, and a player leaving. One flag on the arrival can put the player in a ghost queue, and the listserver then tells the game server how many ghosts are waiting ahead. Kind 1 on the login suppresses that queue message. Everything else from the game server is ignored. Presence, the queue, and the address check are the whole of that link. `[bin]`

The account columns behind a client login are the name, the encrypted password, email, blocked, only-load, admin level, admin worlds, ban time, ban reason, and registration end. `[bin]`

A Windows stand-in that also phones home

The 2003 Windows `ListServer.exe`, signed in the notes to "Rick", is a desktop stand-in. It reads `servers.ini`. A favorite row is sent with a `P` prefix so the client files that server on the gold tab (`tools/server-tools-2001.md`, `recovered-hacks/static-2026-11-01/listserver.md`). `[bin]`

On first start the same image collects the saved classic client logins and hands them to the local browser, aimed at a third-party page. A marker keeps that hand-off to one run. The About box opens a page of its own. The directory behaviour is ordinary. The hand-off is why the notes say not to run the program, packed or unpacked, and why a machine where it has already been started should treat those saved logins as exposed. `[bin]`

A later Reborn-era listserver, the gs2emu snapshot, is source rather than that Windows image (`tools/server-tools-2001.md`). `[src]` Game servers register on one port, current clients connect on another, and pre-2.2 clients use a third. Accounts, server rows, a per-server headquarters record, bans, and guild membership sit in database tables. Passwords there are a salted digest. The project's own lab listserver is a separate Python program, written to answer the clients the lab actually runs (`server-notes/lab-servers-2026-10-30.md`).

What the lab clients receive

Listserver version 2, on loopback, was checked with real clients (`server-notes/lab-servers-2026-10-30.md`). `[lab]`

Client	What it asked	What it got
2.05 rev 2	The older client port, edition only	A list, and then it reached the game
2.17.1	The later client port, account and password in the plain form	The Classic tab, and then the game
2.22	The same later port, the generation that 2.22 uses	Bronze, a status line, Show More, Donations, and then the game

The listserver chooses its reply by login type. It does not check the version token. Whether a build is allowed to play is each game server's own list (`server-notes/public-allowed-versions.md`). `[cap]` `[src]` Two public game servers have printed that list, and only because they were refusing a client: one adventure server allows 2.22 and the build the server table calls 5.07, and one gs2 server allows 2.22 plus a range whose printed ends are 1.41r1 and iPhone Belote 1.5. A server that accepts the client stays silent about the rest of the list. No 5.x or 6.x client in these notes has completed a public login. `[cap]`

The open listservers the lab compared do not all speak every generation. One C# emulator serves a static file and expects a particular login. The 2022 serverlist speaks the generations the lab cares about and names the headquarters tags a 2.22 client draws as tabs. Its file-serving path is commented out, so a game server that asks an open listserver for a global file gets nothing from these trees. `[src]`

The game server that answers second

The early game server is one listen socket and one world. Login, movement, and chat live in the same source file. Levels and NPCs are split out later. Strings are length-prefixed. Accounts are a database, or they are flat files. The cipher of that era, where the notes name one, is the early generation (`server-notes/01-lineage.md`). `[srv]` The generations themselves are the table in chapter 1.

The Reborn-era server in the same lineage runs several worlds from one process, listed in `servers.txt`. A connection is one of three kinds of player: a person, a remote-control session, or an NPC server. Accounts and levels are files, `GRACC001` and `GLEVNW01`. Remote control can browse files and edit options. An NPC-server process is optional. The listserver link has its own message names for register, players, and account checks. `[srv]`

Two packages sit beside that engine and are not the lab's default. Phoenix is the 2.4 binary with a config and a world, listening on the lab game port, and it has no source in the dump. Ash is new code with the same on-disk layout and the same wire rules: several worlds, the three login kinds, the listserver check, levels, baddies, maps, and a relay toward an NPC server. Horses, weapon bytecode, and UPnP are absent from Ash. The full Ash sources stay in the archive named from `ARCHIVAL.md`. `[srv]`

On a script-running server the login itself branches (`server-notes/04-algorithms.md`). `[srv]` The line it reads is the login line in chapter 1. The type on the first line chooses the cipher generation. A client or a later remote control also sends a key. Eight bytes name the build. Then come the account and the password. A build missing from the allowed list is disconnected. A banned address is disconnected. Remote control under a name that is not staff is disconnected. An NPC server is accepted on its own password, told the maps and levels and weapons, and kept. An ordinary player is handed to the listserver for an account check, and the session continues when that check comes back.

The 1999 server cannot answer a question the scripts ask. In a `setcharprop` value, `#n` is the nickname of the player who triggered the script, and `#n` with an id in parentheses picks another player. The property code `#n`, as the first argument, writes the NPC's own nickname (`server-notes/setcharprop-hash-n.md`). `[srv]` GServer13, the 1999 binary and its matching sources, has player and NPC properties and has no `setcharprop` and no GS1 command table. NPC scripts of that year were carried to the client and run there. `[bin]` A loopback of that server with a 2.05 client does not exercise the expansion. A live capture of the server-side expansion is still an open lab. `[lab]`

Why a real 2.05 waits on the loading screen

Stock GServer-v2 does not know the 2.05 token. Mapped into a slot older than 2.1, the login sequence never finishes and the client sits on the loading screen. Real 2.05 waits for the message that names the active level (`tools/lab-setup.md`, `server-notes/lab-servers-2026-10-30.md`). `[lab]` Putting that token in the 2.1 slot lets 2.05 play and costs a real 2.1 client its login. The lab's later beta4 tree gives 2.05 its own slot, sends the active-level message on the old-client path, and sends the level name before the board. With those changes, real 2.05 rev 2, 2.17.1, and 2.22 each list the server, reach the game, and walk. `[lab]`

A few other facts from that same lab belong to the server the player is waiting on. A generation-3 client can put its language line in the same frame as the login. A parser that reads the login and drops the rest of the frame leaves its cipher one step behind, and 2.17.1 then reports an invalid player property. `[lab]` The insertion itself is the generation-3 rule in chapter 1. Files on a hard-coded default list are answered "up to date" without a look at their times, on the older lab build. `[src]` Builds from 2.21 to 2.31 are handed a hidden clientside weapon whose job is to pad chat around a zlib bug in those clients. `[src]` Beta4 moves that repair into the receive path and drops the

weapon. A player who quit while hiding under a carried object can come back invisible unless the account loader clears that hidden bit. The beta4 lab patch clears it. [lab]

The layouts of the login line, the property list, and the board sit in server-notes/03-protocol.md and in protocol/. This chapter uses them as the reason a loading screen ends.

The program that owns the scripts

The 2001 NPC-server document describes a separate program that logs into the game server as an ordinary player and owns the NPCs of a configured set of levels (tools/npcserver-2001.md). [doc] Its loop is a tenth of a second: read from the game server, run the scripts, push changed NPC attributes toward players, push changed player attributes back, and sleep out the rest of the tick. A tenth of a second is the smallest useful sleep. That loop is the script host's clock. The client's frame, where a client was timed, is the 50 ms tick in chapter 2. It loads options, connects, and only then loads the level list and the NPC database.

The game server is described as sending player attributes, temporary level state such as bombs and dropped items, private messages, the time, levels to reload, and world flags. The NPC server sends NPC attributes, player attributes a script changed, replies, weapon grants, and warps. The document names that traffic and does not give packet ids or a port. [doc] NPC Control 1.3, a docs bundle with no executable, says the tool connects to a port on the NewWorld host, that every NewWorld account could connect there, and that the main server required admin level 4 or higher (tools/npc-control.md). [doc]

Scripts run on the server by default. A line //#CLIENTSIDE splits the file: everything after it is shipped to clients, for picture and sound. The document's server-side events include creation, a reload, a player entering, a player leaving the level (a disconnect does not count), a touch, chat, a warp of the NPC, and an explosion. [doc]

A touch needs a rectangle, because the server does not load images. setshape is how a script gives it one. Database NPCs can warp as a player would, which is how recorded police paths are replayed, or wander the overworld and refuse a small link such as a house door. join loads a class from the scripts folder. A class already loaded by another NPC is shared, so editing the class edits every member. Local variables inside with still belong to the caller, and a sleep inside with can leave the swapped player in place. The document calls that a hazard. [doc]

Flags on the player persist in the account. client. flags are also sent to the client, and the client may change them. this. flags belong to the NPC. server. flags are world-wide, shared with other NPC servers of the same world, and never sent to players. server. numbers, by contrast, stay on the one NPC server. [doc]

Admins of that design send private messages to the NPC server's player. Only a nickname carrying (NPC Admin) is answered. The commands add an NPC, reset it, replace a script, fetch a script, replace a class, list database NPCs, dump variables, and reload the level list. Database NPC ids start at 1000. Built-ins sit below that. A script replace cuts a running sleep, so a reset often follows. [doc] The same id split is on the wire in chapter 1: ids below 1000 are level NPCs, and ids of 1000 and above are server-wide.

The 2.05 client already contains many of those names, and it does not treat them all as server orders. [2.05] Some are in the command table with no handler, so they parse and do nothing: the warp queries, join, and the freeze pair. Some run on the client: shape, calling another NPC, tokenizing, player properties, gender, weapons, the level change, explosions, arrays. Some are not script commands in that client at all. A server that wants a 2.05 player to hold still uses its own packets, which the note names, and the script words for freeze do not depend on the packet that announces an NPC server. playerleaves, warpto, and addweapon are absent from the 2.05, 2.17.1, and 2.22 builds that were searched. They look server-only. [checked]

What the scraped scripts actually are

The library in this repo is 3,172 unique scripts taken from classic levels, filed by genre (`npc-library/README.md`). Combat is the largest share, then triggers and visibility, then speech, timers, a miscellaneous pile, doors and chests and signs, and a few dozen warps. The events that dominate the scrape are a player entering, a player touching the NPC, a timeout, the end of a sign, a death, a touch of something else, and a shot (`npc-library/gs1-commands.md`). The commands those events actually call, in the 2.05 table, are text interpreted on the client: the client parses source into a tree and walks it. It does not run a bytecode (`npc-library/gs1-reference-205.md`). `[2.05]` `show` and `hide` are the last two entries of that table. A level feels interactive once a touch can hide an NPC or move the player. That is the offline priority the catalog gives, and it matches the shape of the scrape.

An earlier archive of documented packs, summarized in `npc-library/archival-scripting-notes.md`, reads the same era from the other direction. `[doc]` Across about fifteen hundred scripts the common jobs are quests, level triggers, farms, weapons, shops, minigames, a smaller set of enemy scripts, and HUD overlays. The runtime note there describes pure client-side NPCs, a timeout scheduler, and weapons that persist on the player. Both descriptions are in the repo. The 2001 document is the design of a server that hosts the scripts. The scrape and the pack archive are what levels and later packs actually contain. A scan of 1,420 classic boards, in `npc-library/tile-patterns.md`, is a census of which tiles sit beside which other tiles. It is not a server.

The staff window

Remote control is a Delphi program Stefan Knorr's title line puts his name on. Seven builds from the archives, read from their forms, run from `GSERV014` with seven forms to `GSERV023` with twenty-six (`tools/remotecontrol-lineage.md`). `[bin]` The archive names do not sort by date. The May 2002 build is the newest of that set.

What stays put is the framing: a length, a compressed body, and text messages. Through May 2002 there is no cipher and no key on that socket. The login is a type, the token, the account, and the password. Options live in `controlconfig.txt`. The account and an obfuscated password live in the registry, in the same cipher family the 1.1 guild tool uses for a short code file. The notes record the scheme in `tools/npc-control.md`. The key material stays there.

The start dialog of the early builds asks for a host, a port, the account, and four timers: bush respawn, horse lifetime, how often alignment increments, and how often baddies return. Each timer is sent after login only when its "don't change" box is clear. From August 2001 those timers leave the dialog. From May 2002 the address fields leave too, and the server comes from the list. The 1999 server's own bush wait is the formula in chapter 2. These four boxes are settings the staff client can send. The notes do not record the numbers that were in the boxes.

The forms are the history. The oldest build can list players, disconnect one, edit a player, message one or all, edit accounts, and tell the server to update levels. The account form already carries rights, the look, the position, items, weapons, opened chests, NPC flags, and the combat numbers. August 2000 adds a chat window, server flags, an account list with a filter, and a warp. August 2001 adds rights, comments, and a local ban. May 2002 adds NPC Control, the server list, a file browser, and a short-message popup.

One numbering trap is in the notes so a later server does not trust the later names. In the oldest build, four ids belong to the account dialog. From August 2000 the accounts move, and those four ids sit unused until May 2002 brings them back as server options and folder configuration. A server that still speaks to the oldest remote control cannot use the later names for those four. `[bin]` `[inf]`

Finding the NPC server without being told its address

The May 2002 build is remote control and NPC control in one window (`tools/npc-control.md`). `[bin]` Two sockets: one to the game server, one to the NPC server. A third object talks to the listserver. The NPC login token is `NCL11012`, with the account and password and no key. The same token is still the NPC login in the 2004 build (`tools/remotecontrol.md`). `[bin]`

The program never asks the operator for an NPC-server address. Any player whose nickname contains `(Server)` is treated as one. For each such player the remote control asks for a location. The answer is an address and a port. The remote control opens the NPC socket only if it is not already open, sends the NPC login, and the status bar reads `Connected to NPC-Server`. The loopback address and port baked into the binary are leftovers of an older start dialog. `[bin]` `[inf]` The 2001 private-message interface, aimed at `(NPC Admin)`, is the channel this socket replaced.

The NPC windows are the 2001 commands moved onto that socket: list, add, reset, script, flags, warp, classes, weapons, and the level list. Add NPC still asks for a name, an id, a type, a scripter, a start level, and a position. The type list on the dialog is `OBJECT`, `ANIMAL`, `POLICE`, `HORSE`, `BOAT`, and `HOUSEC`. Class delete is absent in 2002 and present in 2004. `[bin]` The field layouts are in `tools/npc-control.md` and `protocol/npc-control.md`.

By the 2004 build the game-server socket has become the later cipher generation, chosen per flush, while the NPC socket stays on plain compression for the whole session. `[bin]` Folder changes, uploads, and deletes are the file browser. A delete requires the rights string to contain write access. `[bin]` The 2004 note records a staff account already filled in on the form. This chapter does not repeat the password.

Local options that show up across these builds are the staff client's manners: ignore mass messages, strip HTML from private messages, strip images, and append an away marker when the window closes. "Go in away mode" sends the nickname with `- away` on the end. `[bin]`

Guildmaker 1.1, from 1999, is the offline cousin of that stored password (`tools/server-tools-2001.md`). `[bin]` It writes `guildname.code`. A later build can save any `.code` file. The 1.1 client reads the same format for a guild name and for its registration key. The file is 48 bytes, two encryptions of one short block, and both halves have to match or the file is rejected. Samples in the notes decode to a guild name, a script fragment cut at the block length, and the start of a level. The keys stay in that note.

Tools that never log in

A world builder's kit, in the classic set, is a set of windows with no socket in the import table. The Dragon site archive of 2002, among the website packs, lists the same kit as downloads: remote control, the shop, the gif converter, a level generator, a level viewer (`archive/game-assets-websites-2026-11-01.md`). Those programs were not run. The notes below are the programs themselves.

Looks, leaderboards, and a folder of levels

Graal Character Designer 1.2.7, November 1999, Andrew Bridges, and GChar 1.0 from the same author, edit a classic look (`tools/classic-tools/character-designer.md`). `[bin]` `[doc]` Neither imports a socket library. Both expect a Graal folder with `headsconfig.txt` and the head and gif directories. A missing heads file stops them. On startup they find `.chi` files, add characters that are not already listed, and rename those files to `.chr` so the next run does not add them again.

A character file is one text line. The samples share a shape: `CHR:1.00`, a name, a head gif, `male` or `female`, and five colour names. The file does not label the five colours. Several files whose names say Girl are stored as `male`.

The head gif is doing the work. [file] Save to Graal writes the line into the client's list. Snapshot writes a bitmap. Nothing is sent.

gserverstat and gserverstat2 print an HTML leaderboard on standard output (tools/classic-tools/gserverstat.md). [bin] They import no socket call. A shell script, statmaker, runs them forever: one mode, one page, ten seconds between pages, and a long sleep at the end of the cycle. Public pages are kills, deaths, rupees, the two ends of alignment, and online time. Admin pages are the powers. At most fifty rows. An only-load account is skipped, and so is the account whose name is exactly super. The HTML has the nickname and the value. It does not have the password or the account file number. The account file itself is the 1999 blob, a property list plus weapons, flags, and opened chests. [src]

The 1.39 web scripts beside that era do the same job from text files the server already wrote: player count, account count, uptime, and top lists of nickname:value lines (tools/server-tools-2001.md). [src] A PHP sign-up page inserts a row and mails a list of the 1.39 chat commands, from sethead through toall:. A small Linux helper prints a traditional password hash of its argument. It never seeds its random source, so every hash it prints uses one salt. [bin]

GraalLevelGen, December 2000, turns a picture into levels (tools/classic-tools/graal-level-gen.md). [doc] [bin] The operator draws a map whose width and height are 64 times the number of levels across and down, saves a gif or an 8-bit png, and picks a world name. Green is grass and is not drawn. Other colours become forest, water, mountains, flowers, sand, stones, swamp, bushes, and puddles. The mapping is a grid, so the levels are a reading of the picture. The files are named from the world name, through z-99 at the largest, plus a bigmap.txt for the editor's map key. The manual says not to drop that file onto the main world's copy. A second run reads levels that already exist and adds objects. It does not replace the board. That is how the manual stacks trees on mountains. The package says the levels are New World format and that the tool does not apply to 1.3.9 or older. The image contains both New World and older level signatures in a shared editor unit, so which signature a given save writes was not separated out. The file names in the manual are clear. There is no socket. The gmap keys this family of tools writes are the three shapes in chapter 1.

TextConvert walks the current folder and its subfolders and writes New World levels, including links (tools/classic-tools/text-convert.md). [doc] [bin] The image contains the older level signatures and the section words for boards, links, signs, and NPCs, and it fails with strings such as Wrong link data! and Decompression error!. A few strings in the same image say Server error and ask whether you want to quit Graal. They come from a shared editor unit. The import table has no socket library, so those strings are not a connection. The later converter in tools/level-convert/ is the chain in chapter 1, from the oldest level magic through a gmap. TextConvert itself is the folder walk from the older levels to GLEVNW01.

The editor, the shop, and the generators

Until 2.3 the level editor is a mode of the client. In 2.05, F4 switches the game window into that mode and the client refuses the switch while it is online (tools/graaleditor-vs-205.md). [2.05] [GE] 2.17 and 2.22 still carry the editor. 2.31 has lost the editor forms. Its start screen has an Editor button that launches GraalEditor.exe from the game folder, seen live.

GraalEditor 2.3.1.1, from Linux Cyberjoueur, is that same program with the online client removed, shipped as its own executable inside a packaged stub (tools/graaleditor.md, tools/graaleditor-vs-205.md). [GE] There is no list host and no login token. Play still runs a level locally, because the script interpreter stayed. It opens the same level generations the 2.05 editor opens. It wires gmaps into the editor, including level names and a map image and the generated-map keys. It adds a height overlay, predefined NPCs, one NPC window with a script search, and a tiles-array export. The 2.05 gmap reader looks dormant: the strings are in the binary and no

caller was found. [inf] What BOARD looks like in a file this program saved is the open pair in chapter 4. The built-in editor's own feature list, with 2.22 as the reference and every other build as a delta, is tools/level-editor/.

GraalShop is a GANI editor. The name is a Photoshop pun (tools/graalshop.md). Sprite mode is GANI0001, four direction rows on a sheet. Movie mode is GANI0002, with actors that are a character, a sprite, a sound, or text. The file is line-oriented. The shop copies the script and does not run it. On the connection a gani is only the name in chapter 1.

Three generators emit levels and they do not emit the same file (tools/generators.md). TerrainGenerator builds a height field and writes a gmap with seeds and per-level heights, plus levels that are empty boards, a heights overlay, and edge links, and no NPC section. DungeonGenerator stamps an 8 by 11 layout and writes one board, edge links, and NPC blocks. GraalLevelGen, the picture pipeline, emits level-name links. The 2.22 programs are reimplemented as references under tools/generators-222/. The manual limits are recorded with them: the March 2003 manual does not name water, mountain, or sand edge tiles, and the dungeon document does not give the crawler's steps. Feeding one writer's output to another's loader is how a board comes out that will not play. The three GRMAP001 key sets are in chapter 1.

gif2mng, banner date June 2004, writes a PNG for one frame and an MNG for two or more (tools/gif2mng.md). A flag snaps the palette toward the body slots and forces a transparent index. The reference writer in the level-tools notes implements the frame rule on synthetic frames and leaves the slot colours alone, because those colours are not listed in the note.

What the saved worlds contain

The Game Assets archive was listed and sampled. Nothing from it is committed (archive/game-assets-2026-11-01.md, archive/game-assets-worlds-2026-11-01.md, archive/game-assets-graphics-sound-2026-11-01.md). The levels that were opened are the builder's output, sitting in dumps and in archived server folders.

The extension is a weak guide. In the level dumps, hundreds of .nw files are GLEVNW01 and a handful of .nw files are the older GR-V1.03. Two .graal names inside nested archives are New World text. Older magics are rare: one GR-V1.02, one GR-V1.01. An opened New World file is text: the magic, 64 board rows, then links, signs, and NPC blocks. A police file in one dump has four links, one sign, and six NPC blocks. Sign text and scripts were not copied.

Zelda Online's two small archives are the same 36 .zelda files, each Z3-V1.03, dated in the archive as April and May 1998. A later mix keeps three of those files and stores the other .zelda names as GR-V1.03. Zone's gmap archive is nineteen text maps from 2008 through 2014, each GRMAP001 with a width, a height, and a block of level names. Most of them also say not to automap.

Thirteen archived server folders were opened, spread from 1999 file dates to a 2020 starter pack. The 1999 tree is a few hundred GR-V1.03 boards, an image set, and a website folder, with no New World levels, no gmap, and no gani. The 2004 and later trees are GLEVNW01 boards, often a GRMAP001 whose level names are quoted .nw files, and text under names like npcs, weapons, classes, and guilds. One 2006 folder is almost entirely NPC text, thousands of files, with a few hundred levels beside them. Gani files in the folders that have them start at GANI0001, at SPRITE, or once at GANI0002. The same split shows up in the graphics pack's gani archive. Heads in the large heads archive are mostly a strip 32 pixels wide and 560 tall, png and gif and a smaller set of mng, named for a player or an author. [file]

That is the world a builder handed a server: boards, a map that names the boards, pictures, and text scripts. The servers above are what accept a player into that pile.

A 3D server that left the 2D world behind

In February 2000, between the 2D releases 1.32 and 1.33, a side project shipped its own client and its own server (`graal3d/README.md`). `[bin]` Both binaries carry resource stamps of 5 February 2000. The client is `Graal3D.exe`. The server is `GServer.exe`, default TCP port 14901. It does not talk to the 2D game server or to the listserver. Once the server has told each client the others' addresses, position and chat also move directly between clients. A client that has no UDP port gets position and chat relayed over TCP instead.

The server is the 2D server of that year with the game removed. Every Delphi class in the 3D server also exists in the 1999 2D server. The 2D server has the NPC, the baddy, the chest, the horse, the level link, and the classes that record a board change. The 3D server does not. The account object is the same size in both, and accounts live in `accounts.txt` in both. A 2D level object held a 64 by 64 board. The 3D level object holds a name, the file text, who is in the level, and who has already been sent the file. The admin windows carry over, and so does the guild-tag check. The wire codec is the 2D codec, one compressed message per line, with no key byte and no per-packet scramble. The version string is `GR3DV002`. The server also recognises `GR3DV001` and rejects it with a sentence that asks for at least Graal3D beta 2. Positions are 3D, the angle is one byte, and colours are 24-bit. The layouts are in `graal3d/protocol.md`.

The client has no level-script classes, no tiles, and no GANI. The player is eight meshes: body, head, two arms, two hands, two feet. Textures are bitmaps. Levels are DirectX `.x` text files. The connect window offers a nickname, a host, the port, an account, and a password. The nickname default in the form is Stefan. The host default is the loopback address. A guest account is filled in. Hardware options set a fixed UDP port or turn UDP off. A look window edits the meshes and the arm, shoe, and hand colours, and those settings return in `graal3dconfig.txt` (`graal3d/formats.md`). The password in the registry is obfuscated with the same cipher class later 2D clients use to hide a list host. The server rewrites `accounts.txt` when an account changes. A hand-written file of that shape was accepted in the lab. `[cap]` Load-only accounts can log in many times. A normal account that is already in use is refused with that sentence.

Each frame, fifty milliseconds, the client reads TCP, reads UDP, diffs its own properties, and flushes. The step, the jump, the camera, and the face test are the February 2000 body in chapter 2. `[bin]` (`graal3d/client-behaviour.md`).

The lab ran both programs under Wine (`graal3d/lab-2026-10-30.md`). `[cap]` Wine's retained-mode library is mostly stubs, so the real client cannot build faces or player meshes. Two routines were skipped in the running process, not in the file: the client checks its own executable and reports that the Graal execution file is corrupted if the file changes. With that, the real client connects, logs in, sends its look and nickname and UDP port, receives its properties and the level, and saves the level file. It cannot show the world or walk. A reference server written from the notes produced the same message stream as the original for a scripted two-player session, and the real client logged into it and received a byte-identical level file. UDP between two real clients still needs the vendor retained-mode library, which the lab could not fetch.

Leftovers show the port. The player-list disconnect sends the 2D disconnect id, which this client ignores, so that kick arrives silently. The admin message path sends an id the client does show. The server still answers a file request with the 2D raw-data pair, and the 3D client never sends that request. `[bin]`

The years when 3D sat beside the same login

From 2004 to 2007 the client was built on the Torque Game Engine (`torque/README.md`). `[bin]` Torque supplied the console, the script VM, the GUI, the resource manager, the renderer, and, for a 3D server, the UDP netcode. Graal's 2D game and the Graal TCP protocol stayed Graal's. This is a different program from the February 2000 side project. The folder `graal3d/torque-engine-4x.md` is the 4.x plug-in read. `torque/` is the cross-version picture.

A 3D server is two listeners on one host. The list marks it by a `3` prefix on the name. The client strips those two characters, remembers a UDP port one above the listed port, and also opens the ordinary Graal TCP connection to the listed port and logs in as usual (`torque/network.md`). `[bin]` The login is the game-server login in chapter 1. The 3D connection lives only as long as that TCP login. In the lab, the moment the TCP side closed, the real 5.12 client dropped the UDP side and reported that it had been disconnected from the game server. `[run]`

The builds change what "3D" means (`torque/README.md`, `torque/engine-and-formats.md`, `torque/wire-by-build.md`). `[bin]`

Build	What the 3D side is
3.04 Linux, January 2004	Full Torque, game string of the Torque demo, protocol 9, menus in TorqueScript. 3D classes are present. No 3D scripts shipped.
3.153 Mac, February 2005	The same engine, game string `Graal3D`, protocol 19, client scripts including a 3D main script.
4.04r2 Windows, March 2006	A launcher, a 2D engine, and a separate 3D library that is still near-stock Torque: players, vehicles, datablocks. Protocol 19.
4.20r6 Mac, September 2006	Graal's own `GameObject` model. Protocol 21. The stock player and datablock layer is gone.
5.12 Windows, October 2007	One library, still named as the 3D engine, for both modes. `GameObject` plus shape classes for Torque models, interiors, 3D Studio meshes, and text. Protocol 24. Menus are GS2.
5.24 and later	Torque 3D is gone. A GUI class name survives.

The 5.12 handshake was completed live against the real client, answered by a small lab server on loopback (`torque/network.md`). `[run]` The accept carries a table of class names, so a server can pick its own class ids and only needs Graal's names. The game string on that request is `Graal3D`, and the protocol pair is 24 and 24. The account travels in that request in the clear, together with the account password. This chapter does not repeat the password the lab's guest login used. The byte diagram of the handshake stays in that note.

5.12's world objects are script-visible: a ghosted object, a model that plays animations, an interior, a 3D Studio mesh whose collision hulls are closed convex meshes with a naming prefix, text built from a font, and movement that can take an impulse (`torque/engine-and-formats.md`). `[bin]` The 2D player is still the player. A 3D object can carry a gani. Resource files are Torque shapes, interiors, and, in the earlier builds, terrain files. 5.12 streams terrain from the server in chunks, and it wants terrain textures at a fixed size. No 3D server software and no 3D level data are in the archives, so a world on that side would have to be made or found (`torque/README.md`).

3.x menus are compiled TorqueScript. The format, Graal's opcode table, and what the 3.153 scripts do are in `torque/scripting.md`. From 4.x the menus are GS2, the same language family the later 2D client uses. A launcher that offers both clients is how Graal itself did it from 4.x: the list is one list, and the prefix on the name chooses the UDP side.

What the recovered trainers actually do

The trainers in the recovered set type into a classic client window, or they write text into one (`recovered-hacks/behaviour-models.md`). `[bin]` None of them speaks the game protocol. A world checker sees the result: a dig, a use, a walk, a chat line. The waits below are sleeps in the macro, on the machine in front of the client. Copies were unpacked where the packer was intact. No recovered program was executed. A reference module replays the loops inside the test process and does not open a socket.

They share a habit. Several of them look for a top-level window titled `Graal Auto`. The program the other tools call "the key" does not send a game key. It tells the operator they have five seconds to focus the client, sleeps two seconds (the box and the sleep disagree), reads the active window's title, renames that window to `Graal Auto`, and exits. Tools that look for the title do nothing useful until this has run. WaterGet 1.1 does not look for it.

AutoD is a yes/no box and then a forever loop: sleep one millisecond, send `d`. A second box says it was made in 2001 and to stop it from the tray. There is no window check and no chat. `[bin]`

AutoS is three lines: forever, send the two characters `s#`. It sets no key delay and no sleep, and it does not look at a window. `[bin]`

AutoIdle does not chat. One pass sends forty-eight arrow taps and returns to the start. The pass is runs of left and right of different lengths. There is no sleep and no window check. The exact list is in the reference module. Chat rules will not see this one. A world sees a player stepping in place. `[bin]`

WaterGet 1.1 sets the key delay to 300 milliseconds and loops: Down twice, then `a`, then a two-second sleep. It does not check a window title. The bundled howto is the operator's setup, a water can, facing down on a source, the client focused. It is not an extra key. `[bin]`

WaterGet 1.0 is a different loop and it requires `Graal Auto`. It asks for two idle directions, tells the operator to equip a water can, and waits for that window. One pass sends `S` nineteen times with a one-second sleep, waits two seconds, then taps the first direction twice and the second direction twice. If the window is not active it shows a waiting box and sleeps one second. If the window was never renamed it says to use the key first and exits. `[bin]`

Random Walker looks for `Graal Auto`, sleeps two seconds, and exits with the same "use the key first" line if that window is not active. The loop draws an integer from 0 through 5. Zero sends nothing. One through four send one arrow five times. Five sends `S` five times. The key delay is 200 milliseconds. The same waiting box applies when the window is in the background. `[bin]`

Prayer is two macros, and the script body was recovered in the clear (`recovered-hacks/behaviour-models.md`). `[bin]` It requires `Graal Auto`. It asks for three strings. The prompts' examples are a prayer-weapon shortcut, a naginata shortcut, and an idle direction. One pass, while that window is active, sends Tab, types the chat text `/use_skill praying` and Enter three times with two-second sleeps, sends Tab again, sends the naginata shortcut, starts a helper, sends the idle direction once, sends `s` ten times, sends the prayer-weapon shortcut, sends Right, and sends the idle direction twice. The helper sets a key delay, sends Up ten times, and exits. It does not check a window. The praying is a chat line the macro types. There is no other channel.

Graal Load is a Visual Basic window. The label says to put text files beside the program and to type a `load` line in the client. For each line of the chosen file it writes that line into a window it has found, sends the Return key down, and sleeps 150 milliseconds. It does not open a socket. The bundled text file is 164 lines of appearance text, a phrase split across lines, a countdown, and a final shout. The loader's timing does not depend on the words. The file is chat and costume text. `[bin]`

The AFK Bot, by Zorg, imports only the Visual Basic runtime, so the keystrokes go to whichever window the operator focused. The timer is three seconds. The readme says the operator has three seconds after Start to click the client chat box. It then types three phrases, one every three seconds. The three default boxes read `I am AFK`, `I am using an AFK Bot`, and `AFK Bot created by Zorg`. `[bin]` `[inf]` The image stores the three strings. A warning string elsewhere in the set says AFK bots are illegal on some playerworlds (`recovered-hacks/THREAT_TAXONOMY.md`).

The AFK Scroll Bot uses a 100-millisecond timer. A slider labelled Wait Time is how many seconds the operator has to click the chat bar. The start wait is that value times ten ticks, which is the slider in seconds. Once the wait ends, every tick moves the first character of the phrase to the end and submits the line. The default phrase box is `Scroll Text Here`. The shift is in the timer procedure. It is not a form string (`recovered-hacks/remaining-2026-11-01.md`). `[bin]`

Graal Text Sender 1.3.5 looks up a window and can set its text and send a Return. The form offers modes: a word scroller, letters from the middle outward, a line that stays, two words repeating, and letter-by-letter lines. The operator types the phrase. No default phrase is stored. The image declares a key-state check and nothing calls it. There is no phrase list and no log path beside it. It is a chat typer. It is not a key logger. `[bin]`

Three samples stop short of a loop.

The digger's zip inflates a howto and then a partial executable. The howto says to equip a shovel, stand on grass, focus the client, and click ok. It does not name a key or a wait. The executable's compressed stream ends early. What survives includes the name of the macro runtime and smashed pieces of its own words. The script-start marker is absent. There is no chosen key and no wait count, and none is invented from the fragments (`recovered-hacks/remaining-2026-11-01.md`). `[bin]`

The caps-lock utility stays packed. Its version string and its readme say it turns the caps-lock key on and off, and that the command line accepts on or off. The stub does not call through the import table. No socket wording appears. The body was not unpacked by running it. `[bin]`

The miner archive was never read. It was not downloaded.

What a later server still has to choose

The machines above are enough to say what a login is. A directory answers and leaves. A game server checks the build and the account and then owes the client a level, in the order that client's loading screen demands, including the active-level message 2.05 waits for. An NPC server, when a world has one, is a player the staff client finds by nickname. The boards and the scripts are files a builder made offline, and the archived worlds are those files with the extension sometimes lying. February 2000's 3D server is a cousin with the game classes removed. The Torque years keep the Graal login and add a UDP port beside it. The trainers never learned either protocol. They press keys into a window, and a few of them first rename that window so the others can find it.